

FULL STACK DEVELOPER — TECHNICAL ASSESSMENT

Node.js / Express.js • React.js / Vite • SQL

SUBMISSION DEADLINE: 24 SEPTEMBER 2026, 6:00 PM IST

1. Assessment Objective

Build a simplified Multi-Tenant Security Management Platform that demonstrates your ability to develop a secure, maintainable full-stack application. The assessment is designed around real-world requirements relevant to the role, including APIs, SQL, React, JWT, RBAC, multi-tenant data isolation, and security-focused application design.

2. Core Requirements

A. Authentication & Authorization

- Implement JWT-based login and protected APIs.
- Hash passwords securely; do not store plain-text passwords.
- Implement RBAC with three roles: ADMIN, MANAGER, USER.
- Backend must enforce permissions; frontend visibility alone is not sufficient.

B. Multi-Tenancy

- Support multiple organizations/tenants using the same application.
- Users and resources must belong to a tenant.
- A user from Tenant A must never be able to access Tenant B data, even when they know a resource ID.
- Do not trust tenantId or role values supplied by the frontend; derive authorization from the authenticated user.

C. Campaign Management

- Create, list, view, update and delete campaigns.
- Assign users to campaigns and remove users from campaigns.
- Campaign statuses: DRAFT, ACTIVE, COMPLETED, CANCELLED.
- Implement sensible validation for status transitions and other invalid requests.

Example APIs: POST /api/campaigns GET /api/campaigns GET /api/campaigns/:id PUT/PATCH /api/campaigns/:id

D. Security Events & Audit Logs

- Create a security-event module with event type, severity, status, description and timestamp.
- Support filtering/pagination for events.
- Record important actions such as login, campaign creation/update/deletion and user management in an audit log.
- Only authorized users should be able to view audit logs.

E. React Frontend

- Login screen.
- Dashboard showing basic tenant-level metrics such as users, campaigns, open/critical events and recent activity.
- Campaign management screen with search/filter/pagination.
- Security events screen with severity/status filters and pagination.
- User list with role information.
- UI actions should respect the user's role.

F. SQL Database & APIs

- Use MySQL or PostgreSQL.
- Use appropriate relationships, foreign keys, constraints and indexes.
- Filtering, sorting and pagination for large lists must be handled server-side.
- Use parameterized queries/ORM mechanisms to prevent SQL injection.
- Use appropriate HTTP methods, status codes, validation and error handling.

3. Mandatory Security Scenario

Your implementation must prevent cross-tenant access. For example, if Campaign 201 belongs to Tenant B, a user authenticated under Tenant A must not be able to retrieve or modify Campaign 201.

`GET /api/campaigns/201` → must not expose Tenant B data to a Tenant A user

4. README / Engineering Notes

Include a concise README covering:

- Setup and run instructions for frontend, backend and database.
- Environment variables and an `.env.example` file.
- Database setup/seed instructions and sample credentials.
- Short architecture overview and key design decisions.
- Security considerations and how tenant isolation is enforced.
- Brief answers: How would you scale this to 1,000 tenants / 1M users? How would you handle JWT revocation? How would you troubleshoot a production API returning many 500 errors?

5. Optional / Bonus

These are not required and should not come at the expense of the core requirements:

- Docker / docker-compose setup
- Unit or integration tests
- Refresh-token implementation
- Swagger/OpenAPI or Postman collection
- Basic GitHub Actions CI workflow

6. Submission Requirements

- Push the **COMPLETE, RUNNABLE** codebase to a **GitHub repository**.
- Repository should include frontend, backend, database/migrations or schema, README.md and `.env.example`.
- Do not submit only screenshots or partial code. The submitted repository should contain the complete implementation.
- Record a **short DEMO VIDEO** (approximately 5–10 minutes) showing the application running and the main functionality.
- Upload the demo video to Google Drive and provide a shareable link with access enabled for the review team.
- Submit both the GitHub repository link and Google Drive demo video link before the deadline.
- You are expected to understand and be able to explain the submitted implementation.

7. Time & Evaluation

Please prioritize correctness and code quality over implementing every optional feature. A smaller, well-engineered solution is preferred to a large implementation with poor security or architecture.

Submission Deadline: 24 September 2026 • 6:00 PM IST